

Laya Healthcare

Validation Deep Dive

Full Code Walkthrough — PDF 2 of 2

This document explains every notebook cell in full detail — the code, the reasoning, the output, and why each check matters for model quality. For the brief overview, see PDF 1.

Section	Topic	Pg
01	Row Counts	2
02	Null Analysis	2
03	Foreign Key Integrity	3
04	Chronological Integrity	3
05	Distribution Checks	4
06	Claim Type Distribution	5
07	Claim Amount Validation	6
08	Flag Rate Validation	7
09	Label Distribution	8
10	Escalation Logic	9
11	Noise Injection Check	10
12	Feature Stats	11
13	Final Scorecard	12
14	Next Steps & Roadmap	12

01

Row Counts

The warm-up check

The first cell groups all 7 DataFrames into a dictionary and loops through them to print row and column counts in a formatted table.

```
tables = {"users": users, "claims": claims, "app_logs": logs, ...}
for name, df in tables.items():
    print(f"{name:<30} {len(df):>10,} {df.shape[1]:>6}")
```

What each part does

- **tables dictionary** — groups all DataFrames so you loop once instead of repeating the same line 7 times
- **len(df)** — counts rows. The quickest confirmation the table was populated
- **df.shape[1]** — counts columns. Confirms no columns were dropped during generation
- **{count:>10,}** — the comma inside the format string adds thousand separators: 361903 becomes 361,903
- **{name:<30}** — left-aligns in a 30-character field, keeping all columns lined up neatly

The bar chart that follows colours tables with >1,000 rows indigo and smaller ones grey. *app_logs* dominates at 361,903 rows — every user action generates a log entry.

02

Null Analysis

Expected nulls vs problem nulls

Not all missing values are equal. This check classifies every null as either structural (expected by design) or accidental (a data quality problem).

```
def null_report(df, name):
    nr = df.isnull().sum() # count nulls per column
    nr = nr[nr > 0] # keep only columns that have nulls
    if len(nr) == 0:
        print(f"{name}: No nulls")
        return
    for col, count in nr.items():
        pct = count / len(df) * 100
        flag = "Expected" if col in [
            "original_claim_id", "days_since_resubmission",
            "session_duration", "push_notification_opt_in"
        ] else "INVESTIGATE"
        print(f" {col:<40} {count:>8,} ({pct:.1f}%) {flag}")
```

The two types of null

Column	Null %	Why it is expected
original_claim_id	96.5%	Only resubmitted claims (~3.5%) have a parent ID
days_since_resubmission	96.3%	Only populated when resubmission_flag = True
session_duration	27.7%	Login/logout rows are point events with no duration
push_notification_opt_in	10.6%	Not tracked in older app versions — target was 12%

If claim_amount, user_id, or treatment_type appeared with nulls those would be flagged INVESTIGATE — those columns should never be missing.

03

Foreign Key Integrity

Zero orphan rows across all 4 checks

A foreign key is when one table references an ID that lives in another. An orphan row references an ID that does not exist. Even one orphan means feature engineering silently produces wrong results.

```
# Does every claim belong to a real user?
orphan_claims = claims[~claims.user_id.isin(users.user_id)]

# Does every status row reference a real claim?
orphan_status = status[~status.claim_id.isin(claims.claim_id)]

# Does every call reference a real claim?
orphan_calls = calls[~calls.claim_id.isin(claims.claim_id)]
```

How the filter works

claims.user_id.isin(users.user_id) returns True for each claim where its user_id is found in the users table. The ~ (tilde) flips True to False — keeping only rows where the ID was NOT found. These are the orphans. The expected result is len = 0 for all checks.

The full relationship chain

```
users
+-- claims (every claim.user_id must exist in users.user_id)
+-- status (every status.claim_id must exist in claims.claim_id)
+-- logs (every log.claim_id AND log.user_id must exist)
+-- calls (every call.claim_id must exist in claims.claim_id)
```

If calls referenced non-existent claims, the escalation label would be wrong for those snapshots — you would train on incorrect labels.

04

Chronological Integrity

No status timestamp before submission

A claim must be submitted before it can receive a status update. This check joins the status table with the claims table to compare both timestamps on the same row.

```
chron = status.merge(
    claims[["claim_id", "submission_timestamp"]],
    on="claim_id", how="left"
)
violations = chron[chron.status_timestamp < chron.submission_timestamp]
```

Why a merge is needed first

The status table only stores when a status was set — it does not know the claim's original submission timestamp. The merge on claim_id brings both timestamps onto one row so the less-than comparison is possible.

Why violations can happen

The simulation injects a 2% timestamp glitch to simulate DST bugs. Ireland moves from UTC+1 to UTC in late October. The glitch adds or subtracts up to 60 minutes. A claim submitted at 00:05 with 60 minutes subtracted appears at 23:05 the previous day.

Consequence for the model: days_since_submission and time_since_last_status_change would both be negative for violated rows — corrupting two key features.

05

Distribution Checks

Age, archetype, plan type vs YAML targets

Compares actual percentages in the users table against YAML targets for three columns. Draws grouped bar charts with drift annotations above each actual bar.

```
YAML_TARGETS = {
    "age_group": {"18-29":18, "30-44":30, "45-59":27, "60-74":18, "75+":7},
    "plan_type": {"Individual":45, "Family":30, "Corporate":25},
    "behavior_archetype": {"Passive":35, "Engaged":40, "Anxious":18, "Distress":7},
}

for ax, (col, targets) in zip(axes, YAML_TARGETS.items()):
    actual = (users[col].value_counts(normalize=True) * 100).round(1)
    # Grouped bars – YAML left, actual right
    ax.bar(x - w/2, yaml_vals, w, color="#c7d2fe") # pale indigo
    ax.bar(x + w/2, act_vals, w, color=INDIGO) # solid indigo
    # Drift annotation above each actual bar
    diff = actual - target
    color = GREEN if abs(diff) <= 3 else AMBER if abs(diff) <= 5 else RED
    ax.text(..., f"{diff:+.1f}pp", color=color)
```

Key concepts

- **value_counts(normalize=True)** — converts counts to proportions (0.0-1.0), then x100 for percentages
- **x - w/2 and x + w/2** — shifts bars left and right so they sit side-by-side not on top of each other
- **{diff:+.1f}** — the + forces Python to always show the sign: +1.4pp or -1.5pp not just 1.4pp
- **Tolerance: Green ≤3pp, Amber ≤5pp, Red >5pp** — with 2,000 users and random sampling, 1-2pp is normal

Why small drift is expected

Imagine flipping a fair coin 2,000 times. You expect 50% heads but might get 51.3%. The coin is not biased — that is sampling variance. It would actually be suspicious if every column came out at exactly the target percentage.

Column	Largest drift	Result	Note
age_group	+/-1.0pp	PASS	30-44 at 31.4% vs target 30.0%
plan_type	+/-0.3pp	PASS	Closest distribution — very clean
behavior_archetype	+/-1.5pp	PASS	Anxious at 19.2% vs target 18.0%
submission_channel	+5.6pp	NOTE	paper_post over — age-skew effect

06

Claim Type Distribution

Shares and noise variants

Two purposes: verify the share of each claim type matches the YAML, and confirm that capitalisation noise variants are present in the data.

```
def canonicalise(val):
    v = str(val).strip().lower().replace(" ", "_")
    mapping = {
        "gp": "GP", "dental": "Dental", "physio": "Physiotherapy",
```

```
"mental":"Mental_Health", "surgical":"Surgical", ...
}
for key, canon in mapping.items():
    if v.startswith(key[:4]): # match first 4 characters
        return canon
    return val
claims["_tx_canon"] = claims.treatment_type.apply(canonicalise)
```

Why canonicalise is needed

3% of claim types were deliberately written with wrong capitalisation: dental, DENTAL, Physio, SURGICAL. Counting these as separate categories gives a completely wrong distribution. Canonicalise maps all variants back to one standard name before counting.

The first-4-characters trick

```
# "physio" starts with "phys" -> matches "physiotherapy" -> "Physiotherapy"
# "DENTAL" -> lowercase -> "dental" starts with "dent" -> "Dental"
# "gp" starts with "gp" -> matches "gp" -> "GP"
# More robust than exact matching – handles any truncation automatically
```

The noise variant check

```
# Separately list all non-canonical values found
variants = variant_counts[~variant_counts.index.isin(list(yaml_shares.keys()))]
print(variants.to_string())
# This confirms noise was injected – zero variants would mean injection failed
```

07

Claim Amount Validation

Mean, median, outliers per claim type

Checks that money values are calibrated to YAML targets. Uses median not mean for assessment because outlier injection and distributional skew pull the mean away from the target.

```
for tx, target in yaml_amounts.items():
    subset = claims[claims["_tx_canon"] == tx]["claim_amount"].dropna()
    mean_val = subset.mean()
    median_val = subset.median()
    outlier_pct = (subset > mean_val + 3*subset.std()).mean() * 100
    # Use median for drift – it is immune to outliers
    drift_pct = abs(median_val - target) / target * 100
    flag = "PASS" if drift_pct < 15 else "WARN" if drift_pct < 30 else "FAIL"
```

Why median beats mean for this check

Consider GP claims targeted at €75. The 1% outlier injection multiplies some values by 10 — a few claims become €750. That pulls the mean to ~€87 even though 99% of claims are near €75. The median ignores those extreme values.

```
GP example:
Values: 55, 62, 70, 77, 81, 88, 750 (750 is the injected outlier)
Mean: 169 <-- pulled up by outlier
Median: 77 <-- unaffected, close to target of 75
```

The outlier percentage

Any value more than 3 standard deviations above the mean is classified as an outlier. You expect ~1% because that is the injection rate. The `.mean()` on a boolean condition gives the proportion directly because `True=1` and `False=0`.

Surgical: why mean and median diverge

Surgical is modelled with a log-normal distribution — intentional, because real surgical claims have a heavy right tail. Complex inpatient procedures cost far more than day cases. Log-normal distributions always produce mean > median. The €8,707 mean vs €7,662 median is correct expected behaviour.

Type	YAML Target	Actual Median	Drift	Outlier %
GP	75	77	+2.7%	1.1%
Dental	650	647	-0.5%	1.2%
Specialist	700	705	+0.7%	1.0%
Physiotherapy	300	321	+7.0%	1.4%
Mental Health	140	136	-2.9%	0.7%
Surgical	8,500	7,662	-9.9%	0.8%
Maternity	2,200	2,206	+0.3%	0.0%

08

Flag Rate Validation

Missing doc / adjudicator / rejection rates

The three flag columns directly drive escalation probability. This verifies their rates match YAML targets per claim type.

```
for tx, targets in yaml_flags.items():
    subset = claims[claims["_tx_canon"] == tx] # filter to one type at a time
    for key, col in flag_cols.items():
        y = targets[key] # YAML target rate
        a = subset[col].astype(float).mean() # actual rate
        # .mean() on True/False = proportion of True rows
        diff = abs(a - y)
        flag = "PASS" if diff < 0.03 else "WARN"
```

How .mean() gives a rate from boolean columns

```
[True, False, False, True, False]
-> mean = (1+0+0+1+0) / 5 = 0.40 = 40% rate
# True counts as 1, False as 0 -- so mean = proportion
```

Why you filter per claim type first

Surgical has a 22% missing document rate. GP has 5%. If you averaged across all types you get a blended ~11% — which is accurate for neither type and useless.

The Maternity note

Maternity shows 21% actual vs 14% YAML target for missing documents — a +7pp drift. Two causes: small sample (n=205 claims) means variance is higher, and the adjudicator-to-missing-doc interaction compounds slightly more for Maternity. Acceptable for training data.

09

Label Distribution

20.5% positive — 3.9:1 imbalance

The label is the target variable the model predicts. This check confirms it was assigned correctly and that you understand the class imbalance before choosing a model and evaluation metric.

```
label_counts = snaps["label_escalation_48h"].value_counts().sort_index()
label_pct = snaps["label_escalation_48h"].value_counts(normalize=True) * 100
# Text bar chart – visual without a plot
bar = "x" * int(label_pct[label])
# Auto-calculate the XGBoost parameter
print(f"scale_pos_weight = {label_counts[0] // label_counts[1]}") # prints 18
```

What .sort_index() does

value_counts() normally sorts by frequency. sort_index() guarantees the order is 0 then 1 regardless of frequency. Defensive coding — makes the output predictable.

Why class imbalance kills accuracy as a metric

A model that always predicts 0 (never escalates) would be right 79.5% of the time. That sounds impressive but it catches zero actual escalations — completely useless. Accuracy rewards the lazy always-zero model.

```
Metric Why it works for imbalanced data
-----
PR-AUC Measures precision vs recall tradeoff.
Not distorted by the large negative class.
F1 @ 0.3 Lowers threshold from 0.5 to 0.3 to catch more
escalations at cost of more false alarms.
Recall at Only count the recall when precision is still
precision>0.5 acceptable – avoids gaming with low-confidence calls.
```

The daily escalation rate chart

The second chart plots escalation rate by day since submission. Expect: low on day 1 (user just submitted), rising days 3-5 as processing time expectations are exceeded, peak around day 6-10, then falling as most calls have already happened. A flat line means the escalation timing is broken.

10

Escalation Logic Validation

The most important business logic check

This check answers the critical question: are the right users escalating for the right reasons? If the ordering is wrong the training labels encode the opposite of reality.

```
# Step 1: flag which claims generated a call
claims["escalated"] = claims.claim_id.isin(calls.claim_id)
# Step 2: merge with user demographics
merged = claims.merge(
    users[["user_id", "behavior_archetype", "age_group"]], on="user_id", how="left"
)
# Step 3: escalation rate per archetype
arch_rates = merged.groupby("behavior_archetype")["escalated"].mean() * 100
```

```
# Step 4: explicit ordering assertions
assert distress_rate > anxious_rate # must hold
assert age_75_rate > age_1829_rate # must hold
assert rejected_rate > normal_rate # must hold
```

Why a merge is needed first

The claims table knows which claim escalated but not the user's archetype or age. The users table has demographics but not escalation status. The merge on user_id combines both so groupby archetype is possible.

How groupby mean works here

```
merged.groupby("behavior_archetype")["escalated"].mean()
# Groups all claim rows by archetype
# Takes mean of the escalated (True/False) column within each group
# Mean of True/False = proportion = escalation rate
# Distress: 50 escalated of 152 claims -> 0.329 = 32.9%
```

Assertion	YAML driver	Actual result
Distress > Anxious	Distress x2.0 multiplier, Anxious x1.5	32.7% vs 28.5% PASS
75+ > 18-29	75+ multiplier x1.4, 18-29 multiplier x0.75	35.1% vs 14.7% PASS
Rejected > Normal	Rejected flag adds +30pp to call probability	41.2% vs 19.8% PASS

If Passive users escalated more than Distress the model would predict low risk for the most dangerous users — the worst possible outcome when deployed.

11

Noise Injection Check

Confirming intentional imperfections

Confirms all four noise types were actually injected at configured rates. This is not looking for problems — it confirms test conditions for your preprocessing pipeline are in place.

```
# 1. Claim type variants – confirm ~3% non-canonical values
variants = [t for t in all_types if t not in clean_types]
variant_pct = len(claims[claims.treatment_type.isin(variants)]) / len(claims) * 100

# 2. Region variants – confirm ~4%
region_variants = region_vc[~region_vc.index.isin(clean_regions)]
variant_region_pct = region_variants.sum() / len(users) * 100

# 3. Session duration nulls – ONLY on session events, not login/logout
session_events = logs[logs.event_type.isin([
    "claim_status_view", "document_upload", "in_app_chat_start"
])]
null_sess_pct = session_events.session_duration.isnull().mean() * 100

# 4. Outliers – beyond 3 standard deviations within each claim type
outlier_pct = (subset > subset.mean() + 3*subset.std()).mean() * 100
```

Why session events are filtered first

Login and logout rows always have null session_duration — they are point events not sessions, so there is no duration to record. Including them would inflate the null rate unfairly. The 8% YAML target applies only to rows where duration should exist but was lost due to old app version tracking gaps.

What these noise injections test in your pipeline

Noise type	Rate	What it tests
Claim type variants	~3%	Your normalisation step — lowercase + canonical map
Region name variants	~4%	Address standardisation — Co. Dublin, LK, etc.
Session duration nulls	~8%	Imputation strategy — median per event_type bucket
Claim amount outliers	~1%	Winsorisation — cap at 99th percentile before model

12

Feature Stats and Histograms

Distribution and predictive signal

Two purposes: confirm features have sensible ranges, and preview which features separate the two classes before building the model.

```
stats = snaps[numeric_features].describe(percentiles=[.25, .5, .75, .95]).T
# .T transposes so features are rows, statistics are columns
# Easier to read than 12 statistics-as-rows layouts
```

What to look for

- **min less than 0** — impossible for count features, would indicate a calculation bug
- **max unrealistically high** — days_since_submission max=14 is correct (lifecycle capped), login max=19 is plausible for Distress

- **mean much larger than P50 (median)** — signals right skew, these features need log-transform
- **std greater than mean** — also confirms skew — claim_amount std=4,457 vs mean=2,006

Features requiring log-transform (mean >> median)

Feature	Mean	Median	Ratio	Transform
claim_amount	2,006	535	3.75x	log1p(claim_amount)
relative_claim_cost	5.03	1.34	3.75x	log1p(relative_claim_cost)
behavior_acceleration	2.10	0.74	2.84x	log1p(behavior_acceleration)
status_views_24h	7.56	2.00	3.78x	log1p(status_views_24h)

Why log1p not log

```
import numpy as np
np.log(0) # returns -inf -- breaks for zero values
np.log1p(0) # returns 0 -- safe because log(1+0) = log(1) = 0
# login_count_24h can be 0 (user did not log in that day)
# You MUST use log1p or features will contain infinite values
```

Reading the overlapping histograms

For each feature two overlapping distributions are drawn — amber for escalation cases, indigo for non-escalation. The key question: do the two distributions sit in different places on the x-axis?

```
ax.hist(non_escalated, bins=40, color=INDIGO, alpha=0.6, density=True)
ax.hist(escalated, bins=40, color=AMBER, alpha=0.7, density=True)
# density=True normalises both to the same total area
# Without it the 18x larger non-escalation group would visually dominate
```

density=True is essential. Without it the non-escalation histogram would be 18 times taller simply because there are 18 times more rows. density=True rescales both so areas equal 1 — making shape comparison fair regardless of class size.

13

Final Scorecard

11 PASS 3 NOTE 0 FAIL

The final notebook cell prints a single consolidated summary of all 14 checks. One screenshot to share with your team. PASS means clean — nothing to action. NOTE means worth knowing but not a blocker. FAIL means fix before training.

Check	Result	Detail
Row Counts	PASS	All 7 tables populated
FK Integrity	PASS	0 orphan rows across all 4 checks
Chronological Order	PASS	0 status timestamps before submission
Age Distribution	PASS	Within +/-1pp of YAML
Plan Type	PASS	Within +/-0.3pp of YAML
Archetype	PASS	Within +/-1.5pp of YAML
Claim Type Shares	PASS	Within +/-2.5pp of YAML
Claim Amounts	PASS	Medians within 15% of targets
Flag Rates	PASS	Within +/-2pp, Maternity +7pp noted
Noise Injection	PASS	All four noise types confirmed
Escalation Logic	PASS	Distress > Anxious, 75+ highest
Label Distribution	NOTE	3.9:1 imbalance — use scale_pos_weight=4
Channel Distribution	NOTE	paper_post +5.6pp — age-skew working correctly
Maternity Miss Doc	NOTE	21% vs 14% — small sample n=205

14

Next Steps and Full Roadmap

Preprocessing, features, modelling

Preprocessing — do these first, before any feature engineering

Normalise treatment_type and region	Lowercase + strip + regex map to canonical names. Must happen before groupby or encoding.
Winsorise claim_amount and relative_claim_cost	Cap at 99th percentile. Prevents tree splits being dominated by the 1% outlier rows.
Impute days_since_resubmission	Fill 0 when resubmission_flag = False. The null is already captured by the flag itself.
Impute session_duration	Median per event_type bucket — login rows get a different median to chat session rows.

Feature Engineering — derive new signals from existing columns

log1p transforms	Apply to claim_amount, behavior_acceleration, relative_claim_cost, status_views_24h.
------------------	--

stress_score composite	Weighted sum: missing_doc + rejected + adj_flag + resubmit. Captures compound risk.
channel_risk_flag	1 if paper_post, else 0. Paper users cannot self-serve claim status checks online.
age_paper_interaction	1 if age 75+ AND paper_post. Highest risk combination — captures the interaction directly.

Modelling — algorithm, split strategy, evaluation metrics

Time-based train/test split	Train June-September, test October-November. Random split leaks future patterns into training.
Baseline: Logistic Regression	Use class_weight='balanced'. Fast, interpretable, gives a benchmark to beat.
Primary: XGBoost or LightGBM	Set scale_pos_weight=4. Use early_stopping_rounds to prevent overfitting.
Evaluation metrics	PR-AUC as primary. F1 at threshold 0.3. Recall at precision > 0.5. Never accuracy.

This completes the full deep-dive of the Laya Healthcare claim escalation validation notebook. Dataset is validated and ready for preprocessing and model training.